Embeddings

["cat, dog, kitten, puppy]

SQL

TRADITIONAL DATABASE

Cat

Result: [cat]

Exact Matches.

["cat, dog, kitten, puppy]

Embeddings

VECTOR DATABASES

Cat

Text → [ ]

Apple → [0.2, 0.3, 0.5]

⇓

Vector representation

#Find Similar Meanings

results = {Cat, Kitten}

Embedding Models → Huge Amount Data.

⇓

Vector Representation.

OpenAI, Huggingface

Cat, Kitten

[0.2, 0.3, 0.4]

[0.2, 0.4, 0.3]

} => Cosine Similarity

Feature Representation

Marvel

		Action	Comedy	Suspense
A	Iron Man	[0.95	0.2	0.6]
B	Hulk	[0.96	0.4	0.7]
C	Sherlock Holmes	[0.6	0.75	0.9]

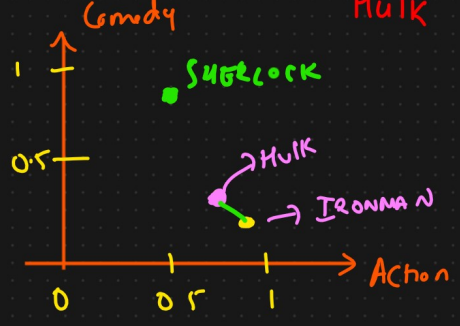
Cosine Similarity = $\frac{A \cdot B}{\|A\| \|B\|}$

Netflix

IronMan

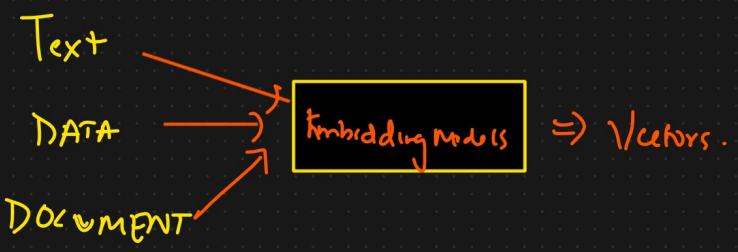
⇓

Hulk



Embedding Model

[OpenAI , Huggingface]



Vector Store Vs Vector Databases

Vector Store

What it is

A lightweight library or tool focused on storing and searching vectors efficiently

Core Function

Simple similarity search - find the K nearest neighbors to a query vector

Architecture

Usually runs in-memory or as a local file, single-machine operation

Best For

Prototypes, research, small applications with <1M vectors

Vector Database

What it is

A full-featured database system designed for vector data at scale

Core Function

Advanced search with filters, metadata queries, and database operations

CRUD

Architecture

Distributed system with replication, sharding, and high availability

Best For

Production systems, enterprise applications, billions of vectors

Quick Comparison

SCALE		SETUP TIME		COST	
~1M vectors	Billions+	Minutes	Hours/Days	Free/$	$$$/$$$$
QUERY SPEED		FEATURES		DEPLOYMENT	
Microseconds	Milliseconds	Basic Search	Full CRUD	Local	Cloud

Popular Examples

Vector Stores

FAISS · Annoy · ChromaDB · ScaNN · NMSLIB

Vector Databases

DATASTAX

Pinecone · Weaviate · Qdrant · Milvus · Vespa

Use Vector Store When

- → Building a proof of concept
- → Working with < 1 million vectors
- → Need fastest possible search speed
- → Have limited budget
- → Want full control over the implementation
- → Building embedded applications

Use Vector Database When

- → Building production applications
- → Need to scale beyond millions of vectors
- → Require high availability & reliability
- → Need advanced filtering & metadata search
- → Have multiple users/tenants
- → Want managed infrastructure

The Simple Rule

Start with a Vector Store for prototyping and learning. Graduate to a Vector Database when you need production-scale features, reliability, and advanced querying capabilities.

Enhanced RAG Techniques

Advanced Chunking And Preprocessing

pdf, txt, word, json
⇓
Document

Document Splitter
Recursive Character

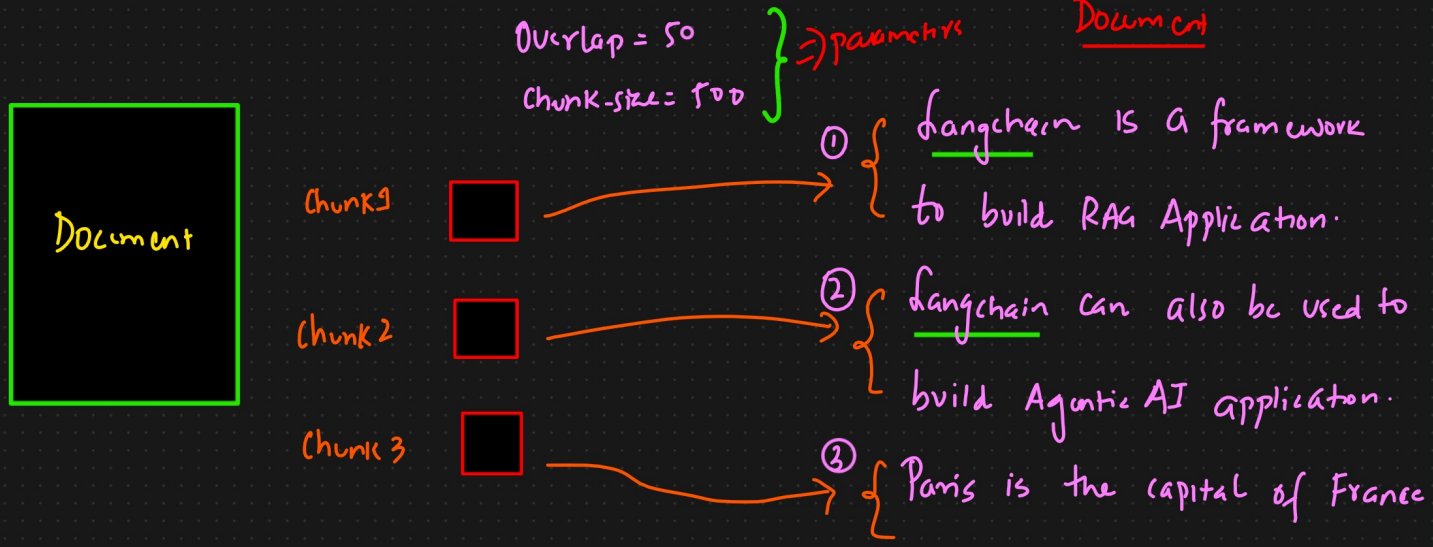
1) Semantic Chunking

Semantic chunking is the process of splitting a document into meaningful units (chunks) based on semantic similarity — not just by number of tokens or lines.

This is important in RAG systems because:

Better chunks → better retrieval → better grounding → better answers.

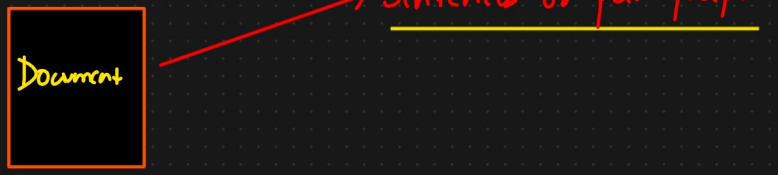
Chunks should be self-contained, contextually rich, and logically separated.



Contextual rich logically separated chunks.

How does It Work ?

1) Document Segmentation



2) Sentence Embedding : Each sentence is converted into a vector representation

OpenAI, Hf => Sentence Transformers

3) Semantic Similarity Check : Cosine similarity between adjacent embedding

S1 ↔ S2 => Similar => 0.95

S2 ↔ S3

S3 ↔ S4

{ Threshold => 0.8
=> 0.75
=> 0.95 }

4) Merging Of Sentences : Merge adjacent sentences if they are semantically similar

5) Form chunks : Merge S1 and S2 => [S1, S2] => chunk 1

=> [S3] => chunk 2.

Chunks

1. LangChain is a framework for building LLM-powered apps.
2. It integrates with tools like OpenAI and Pinecone.
3. The Eiffel Tower is located in Paris.
4. France is a popular tourist destination.

⇓ Semantic chunking

Op

["LangChain is a framework...", "It integrates with tools..."]
["The Eiffel Tower is located in Paris."]
["France is a popular tourist destination."]

Hybrid Search Strategies

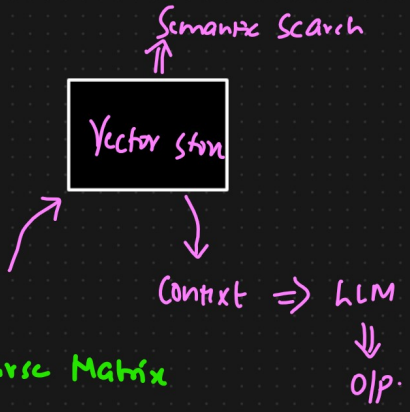
1) Combining Dense And Sparse Retrieval

Hybrid retrieval combines both dense and sparse scores (e.g. using a weighted sum or learning-to-rank methods) to improve recall and relevance.

You get the semantic power of embeddings, and the exact match of keywords — best of both worlds.

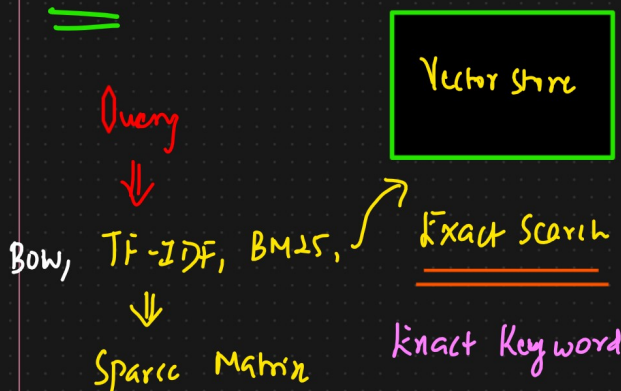
Doc ID	Content
D1	"LangChain supports agents and chains"
D2	"The Eiffel Tower is in Paris"
D3	"You can build LLM-powered apps using LangChain"

RAG



Text → Sparse Matrix

NLP



Exact Keyword Search

① Sparse Retrieval : Matches exacts words using techniques such as TF-IDF

⇓ Exact Search

Sparse Matrix

I want to have food => [1 0 0 0 1]

② Dense Retrival => Semantic Meaning Using Vector Embedding

FAISS, CHROMA DB



→ Semantic search

=> O/P Similar Matching Sentences

{ Exact Search [Sparse Matrix] + Semantic Search [Dense Matrix] ← } Hybrid Search

⇓

Retrieval

1) Dense Retrival (Embeddings + Cosine Similarity)
+
2) Sparse Retrival (TF-IDF).

$$Score_{hybrid} = \alpha * score_{dense} + (1-\alpha) * score_{sparse}$$

$Score_{dense}$ = Cosine Similarity (Inp, Vectorstore)

$Score_{sparse}$ = TFIDF (Inp, Vector Representation)

α = Weightage = 0.5

Doc ID Text
D1 "LangChain helps build LLM apps"
D2 "Pinecone is used for vector search"
D3 "Eiffel Tower is in Paris"

Your query: "build application using LLM"

Ip: Build application using LLM

Cosine Similarity [Dense Embedding]

D1	0.85
D2	0.40
D3	0.10

BOW, TF-IDF, BM25

D1	0.60
D2	0.20
D3	0.10

⇒ Sparse Matrix.

$\alpha = 0.5$

$D1 = 0.5 \times 0.85 + 0.5 \times 0.6 = \approx 0.82$

$D2 = 0.5 \times 0.40 + 0.2 \times 0.5 = \approx 0.18$

$D3 = 0.5 \times 10 + 0.3 \times 0.5 = \approx 0.04$

1

Hybrid Search Strategies

10 recorder

RAG pipeline

2) Re-Ranking Techniques

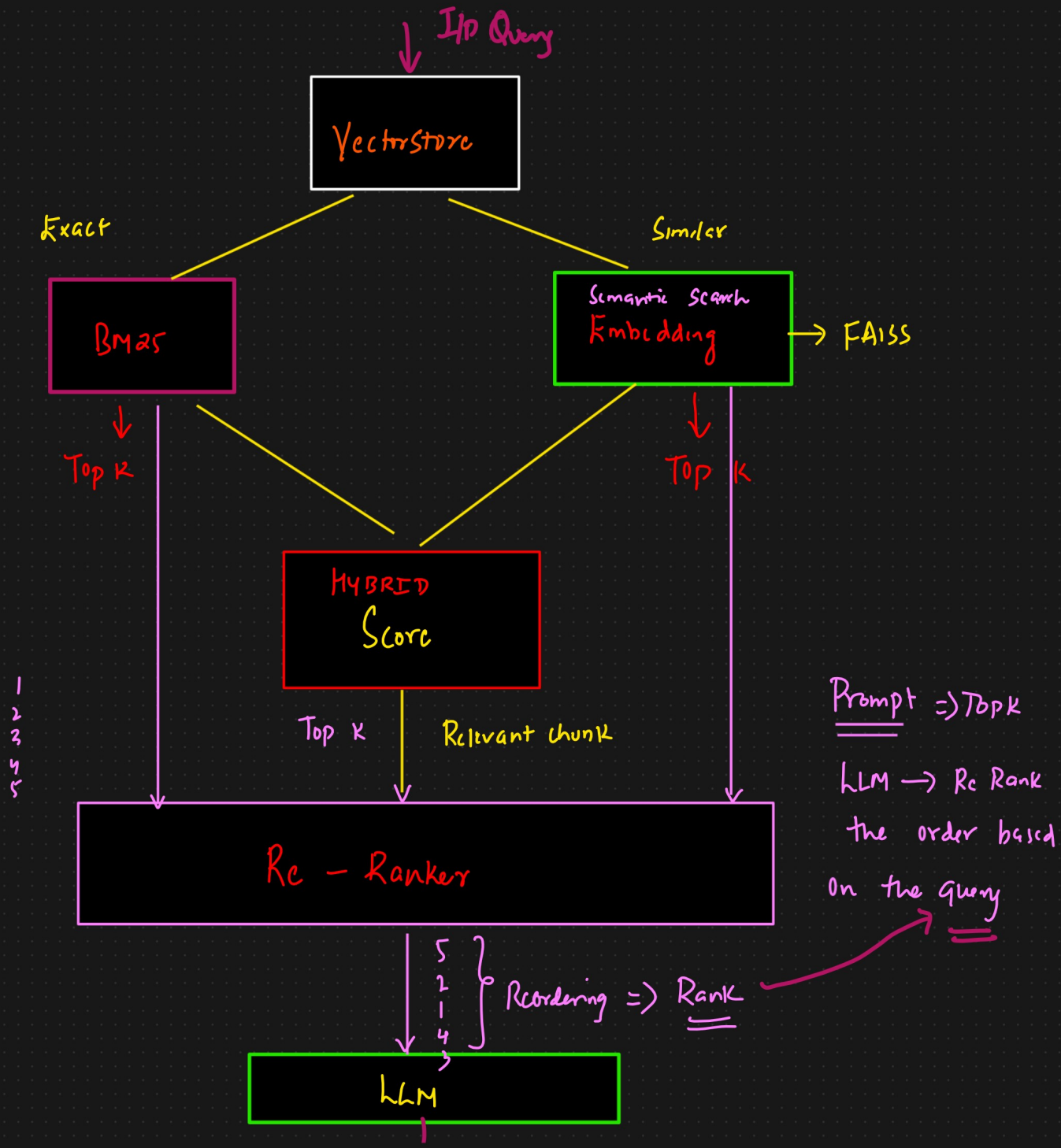
1)
2)
3)

Re-ranking is a second-stage filtering process in retrieval systems, especially in RAG pipelines, where we:

1. First use a fast retriever (like BM25, FAISS, hybrid) to fetch top-k documents quickly.

2. Then use a more accurate but slower model (like a cross-encoder or LLM) to re-score and reorder those documents by relevance to the query.

👉 It ensures that the most relevant documents appear at the top, improving the final answer from the LLM.



Stages

1) Retrieval
2) Re-Ranking
3) Generation

↓

O/P => Accurate

Practical

📊 Why Use Rerankers in a RAG Pipeline

🔍 Reason	💡 Without Reranker	✅ With Reranker
1. Relevance of Context	Top-k documents may be loosely or partially related	Top-k documents are re-scored and reordered for maximum relevance
2. Factual Accuracy	LLM may hallucinate if poor context is retrieved	Irrelevant docs are filtered out → grounded, factual answers
3. Handling Ambiguity	Retriever lacks deep understanding of query intent	Reranker evaluates full query–doc pair → better intent alignment
4. Semantic Matching	Dense retrievers may miss low-similarity but relevant docs	Reranker uses deeper models (cross-encoders / LLMs)
5. Keyword vs Meaning	BM25 may favor exact match even if not meaningful	Reranker balances semantic and lexical relevance
6. Prioritization of Evidence	All retrieved docs treated equally	Most relevant documents float to the top
7. Long-Tail Queries	Weak retrievers struggle with uncommon queries	Rerankers better capture rare but meaningful matches
8. LLM Efficiency	Poor context leads to verbose or incorrect answers	High-quality context improves precision and conciseness
9. Noise Reduction	Noisy docs (ads, unrelated text) may slip in	Reranker pushes noisy docs to the bottom or filters them out
10. Flexible Scoring Strategies	Fixed retriever scoring	Reranker can include metadata, recency, user preferences